Written Homework Assignment #2

Problem 5.1 (Stephens, Page 116)

Difference between Component-Based Architecture and Service-Oriented Architecture:

- **Component-Based Architecture (CBA):** A software design approach where the system is structured using reusable and interchangeable components. Each component encapsulates a specific functionality and interacts with others through well-defined interfaces. This promotes modularity and reusability.
- **Service-Oriented Architecture (SOA):** A design paradigm where applications are composed of loosely coupled services that communicate over a network, usually using web protocols like REST or SOAP. SOA enables distributed computing and is ideal for integrating heterogeneous systems.

Key Differences:

- **CBA** focuses on modular software design within a single system, while **SOA** is oriented toward distributed computing across systems.
- **CBA** components are tightly integrated, whereas **SOA** services communicate via APIs and are more loosely coupled.
- **SOA** is typically used in enterprise applications, while **CBA** is common in application and UI development.

Problem 5.2 (Stephens, Page 116)

Most Appropriate Architectures for a Tic-Tac-Toe Phone App (Single Player, No External Database):

- **Component-Based Architecture:** Since the game is simple and self-contained, a modular approach where UI, game logic, and AI components are separate would work well.
- **Model-View-Controller (MVC) Pattern:** Helps separate the game logic (Model), user interface (View), and user interactions (Controller).
- **Why?**
    - No need for external service calls.
    - Lightweight and efficient for a mobile application.
    - Facilitates code reusability and maintainability.

Problem 5.4 (Stephens, Page 116)

Appropriate Architectures for a Chess Game with Online Multiplayer:

- **Service-Oriented Architecture (SOA):** Since the game requires network communication for online play, SOA enables scalable interaction between game clients and servers.
- **Client-Server Model:** The game logic runs on a central server, while users interact through client devices.
- **Why?**
    - Enables real-time multiplayer interaction.
    - Allows for storing match history and player statistics.
    - Scales better as more users join.

Problem 5.6 (Stephens, Page 116)

Database Structure and Maintenance for ClassyDraw Application:

- **Database Type:** Relational Database (e.g., PostgreSQL) or NoSQL (e.g., Firebase Firestore) depending on scalability needs.
- **Structure:**
    - **Users Table:** Stores user information and authentication.
    - **Drawings Table:** Stores vector graphics data (shapes, colors, coordinates).
    - **Versioning Table:** Keeps track of edits and changes for version control.
- **Maintenance:**
    - Regular backups.
    - Indexing for performance optimization.
    - User access control to prevent unauthorized modifications.

Problem 5.8 (Stephens, Page 116)

State Machine Diagram for Parsing Floating-Point Numbers in Scientific Notation: The state machine should recognize patterns like:

- `+37`
- `-12.3e+17`
- `5E-5`

States:

1. **Start:** Accepts an optional `+` or `-` sign.
2. **Integer Part:** Accepts digits.
3. **Decimal Point:** Accepts a `.` (optional).
4. **Fractional Part:** Accepts digits after the decimal.
5. **Exponent Indicator:** Accepts `e` or `E`.
6. **Exponent Sign:** Accepts optional `+` or `-` after `e` or `E`.
7. **Exponent Digits:** Accepts digits for the exponent value.
8. **Final State:** A valid number is recognized.

Problem 6.1 (Stephens, Page 138)

Properties of ClassyDraw Classes (Line, Rectangle, Ellipse, Star, Text):

- **Shared Properties:**
  - `position` (x, y coordinates)
  - `color`
  - `outline`
    - `strokeWidth`
  - `rotation`
  - `opacity`
- **Non-Shared Properties:**
  - **Text** has `fontSize`, `fontStyle, message`
  - **Rectangle** has `width`, `height`
  - **Ellipse** has `radiusX`, `radiusY`
  - **Star** has `numPoints`, `innerRadius`, `outerRadius`
  - **Line** has `startPoint`, `endPoint`
- **Partial Sharing:**
  - **Rectangle and Text(box)** share `width` and `height`.
  - **Ellipse and Star** both have a smaller and larger radius which will be denoted `radiusA, radiusB`
- **Implementation Strategy:**
  - **Shared properties** should be in a base class like `Drawable`.
  - **Unique properties** should be implemented in subclasses.

Problem 6.2 (Stephens, Page 138)

Inheritance Diagram for ClassyDraw Classes:

